Software Engineering

Course 5 – 24 March 2025

adiftene@info.uaic.ro

Content

- From previous courses…
- GRASP
  - Low Coupling
  - High Cohesion
- SOLID and Other Principles

RE

- Why do we need modelling?
- How can we model a project?
- SCRUM – roles, values, artifacts, events, rules

GRASP

- GRASP = General Responsibility Assignement Software Patterns (Principles)
- Described by Craig Larman in the book *Applying UML and Patterns. An Introduction to Object Oriented Analysis and Design*
- **It helps us to assign responsibilities to classes and objects in the most elegant way possible**
- Examples of principles used in GRASP: *Information Expert* (or Expert), *Creator*, *High Cohesion*, *Low Couplig*, *Controller* Polymorphism, Pure Fabrication, Indirection, Protected Variations

What responsibilities?

- To do:
  - To do something himself, as well as creating an object or make a calculation
  - Initialization of an action in other objects
  - Controlling and coordinating other objects

- To know:
  - Private attributes
  - Its own objects
  - The things you can do or those you can call

(Design) Pattern

- Also known as: template, model
- It is a general solution to a common problem
- Each pattern has a catchy and meaningful name (eg. Composite, observer, iterator, singleton, etc.)

Information Expert 1

- **Problem:** to which classes a given behavior (operation) has to be assigned?

- **A better allocation of operations leads to systems that are:**
  - Easy to understand
  - More easily extended
  - Reusable
  - More robust

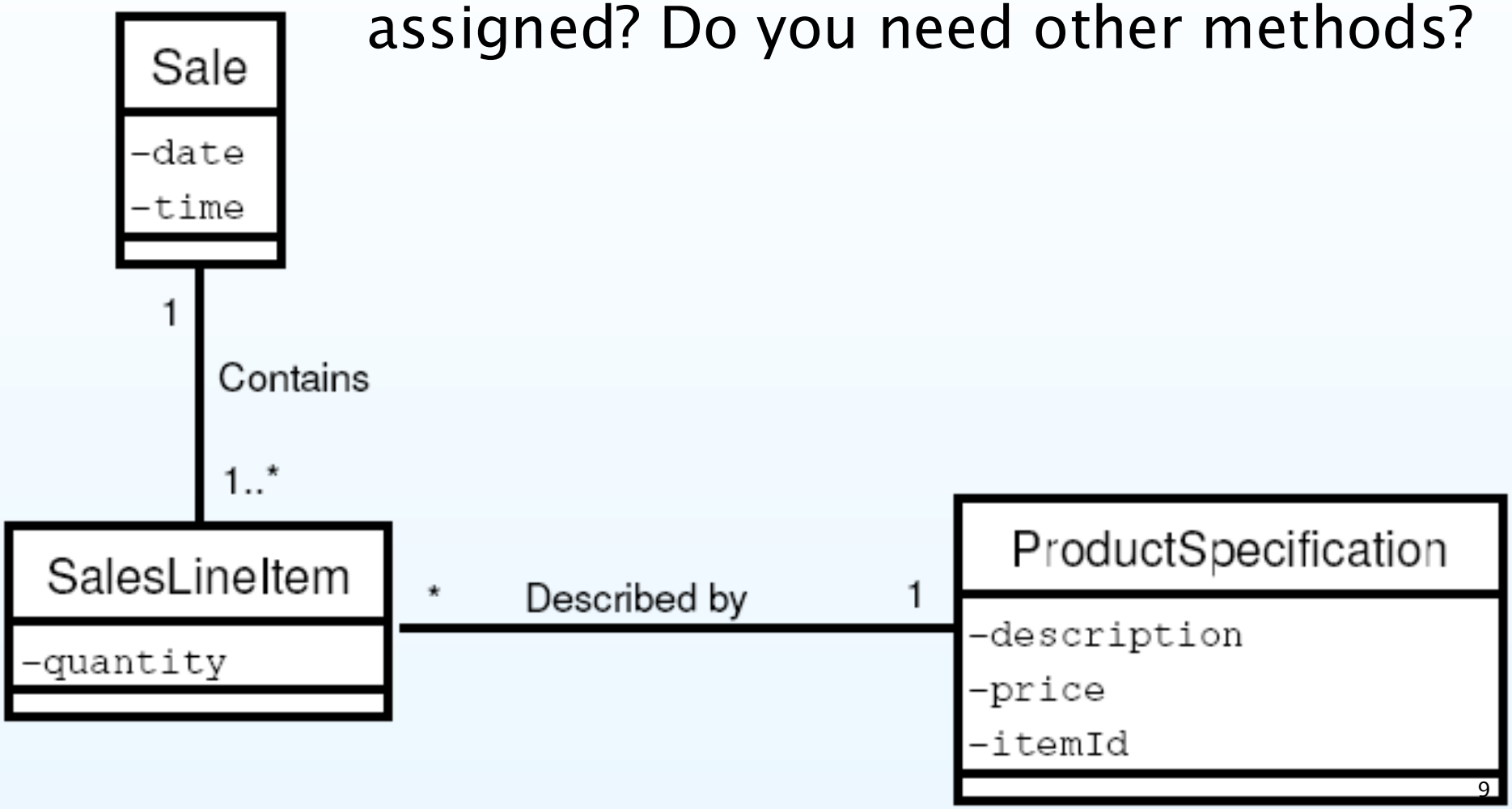
Information Expert 2

- **Solution**:
- Start assigning responsibilities to a class that has the information necessary to fulfill those responsibilities

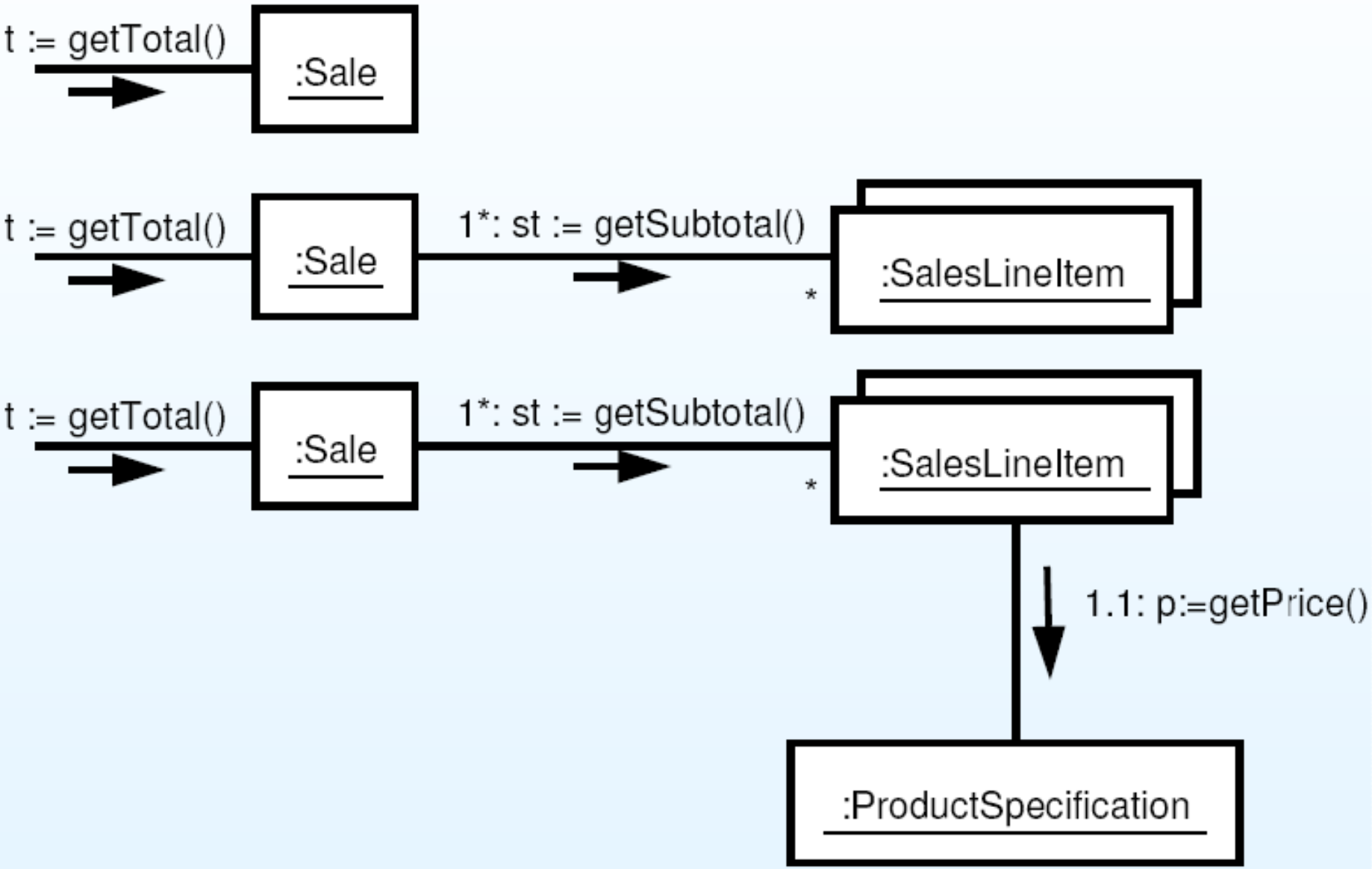
- **Recommendation:**
- start assigning the responsibilities clearly highlighting them

Example 1

- To which classes should method getTotal() be assigned? Do you need other methods?

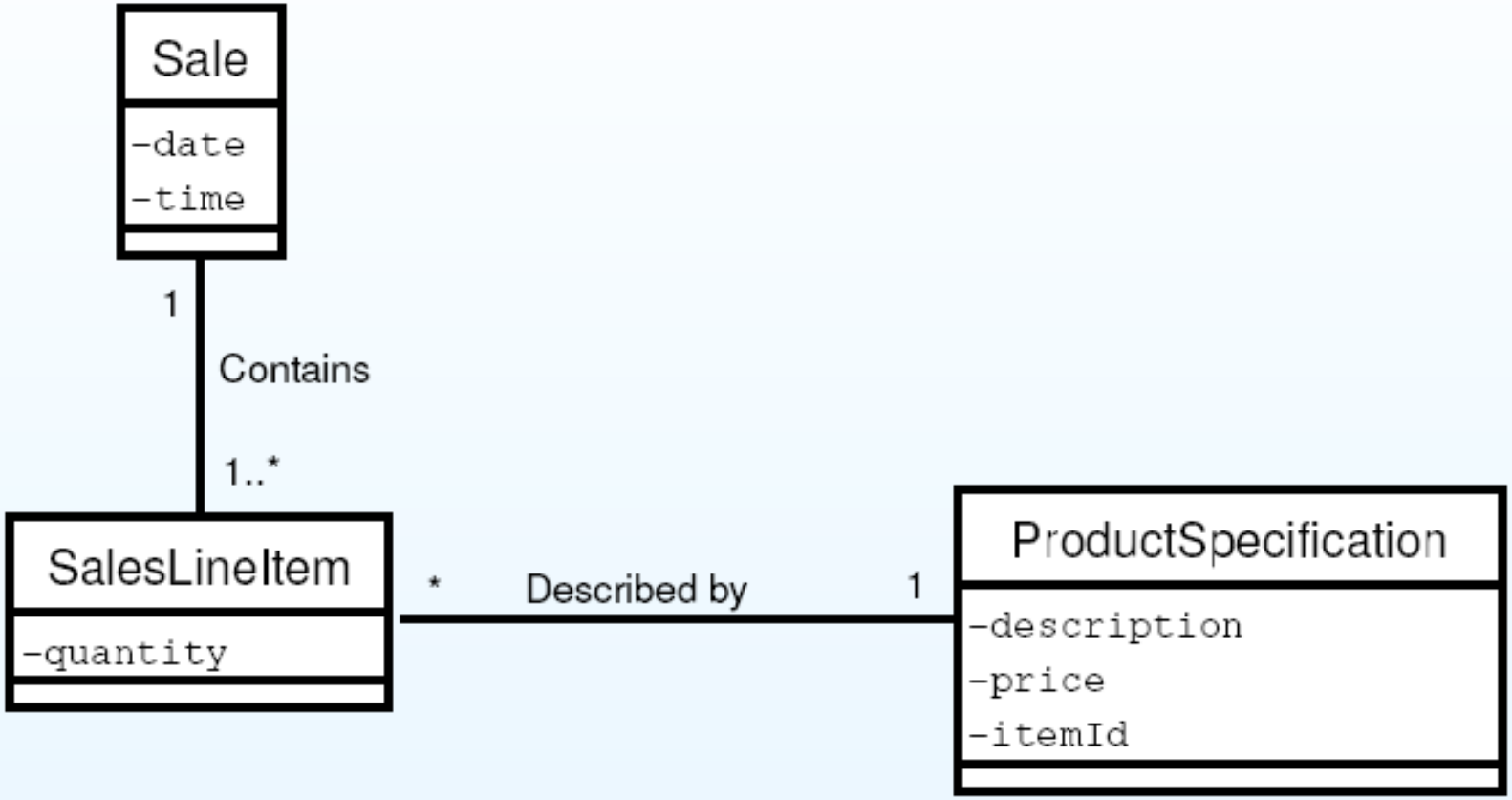


Example 2

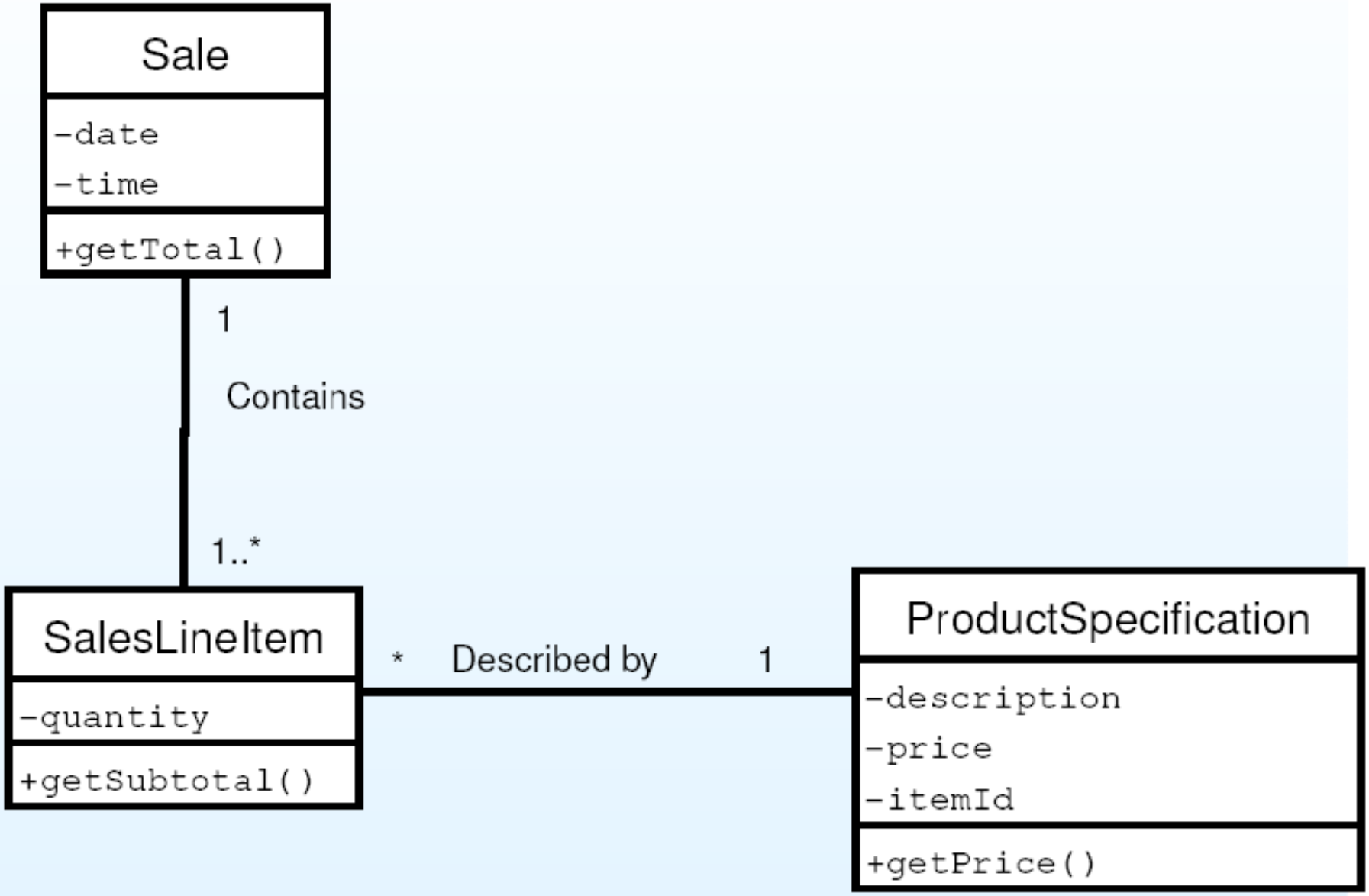

t := getTotal()
:Sale
t := getTotal()
:Sale
1*: st := getSubtotal()
*
:SalesLineItem
t := getTotal()
:Sale
1*: st := getSubtotal()
*
:SalesLineItem
1.1: p:=getPrice()
:ProductSpecification

Possible solution 1

Class	Responsibilities
Sale	to know the total value purchase
SalesLineItem	to know the subtotal for an item
ProductSpecification	to know the price of the product



Possible solution 2

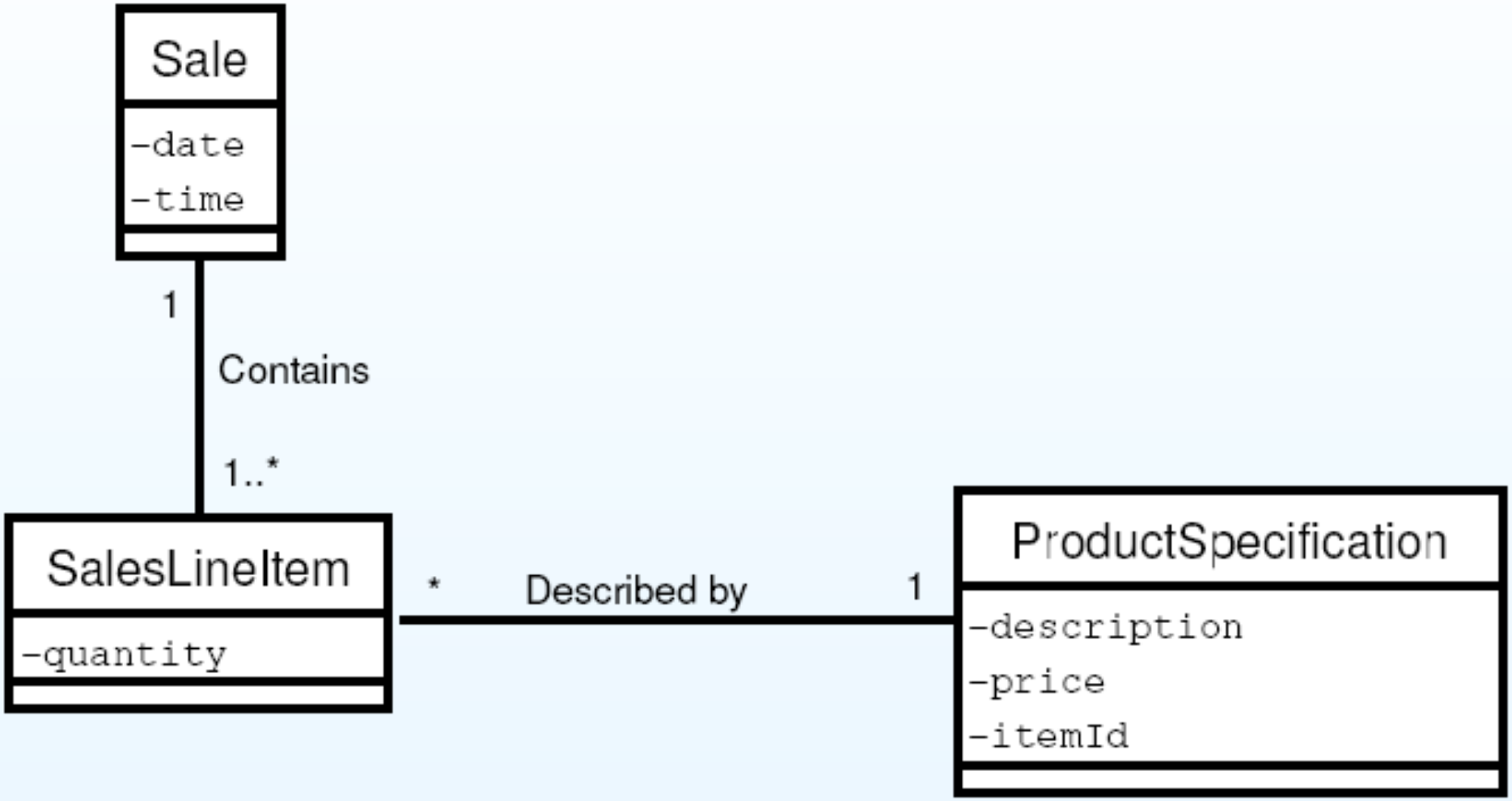


Creator 1

- **Problem:** Who should be responsible for creating an instance of a class?
- **Solution:** Assign class B the responsibility to create instances of class A only if at least one of the following is true:
  - B aggregates objects of type A
  - B contains objects of type A
  - B uses objects of type A
  - B has initialization data to be transmitted to instantiate an object of type A (B is therefore an expert in terms of creating objects of type A)
- **Factory pattern** is a more complex variant

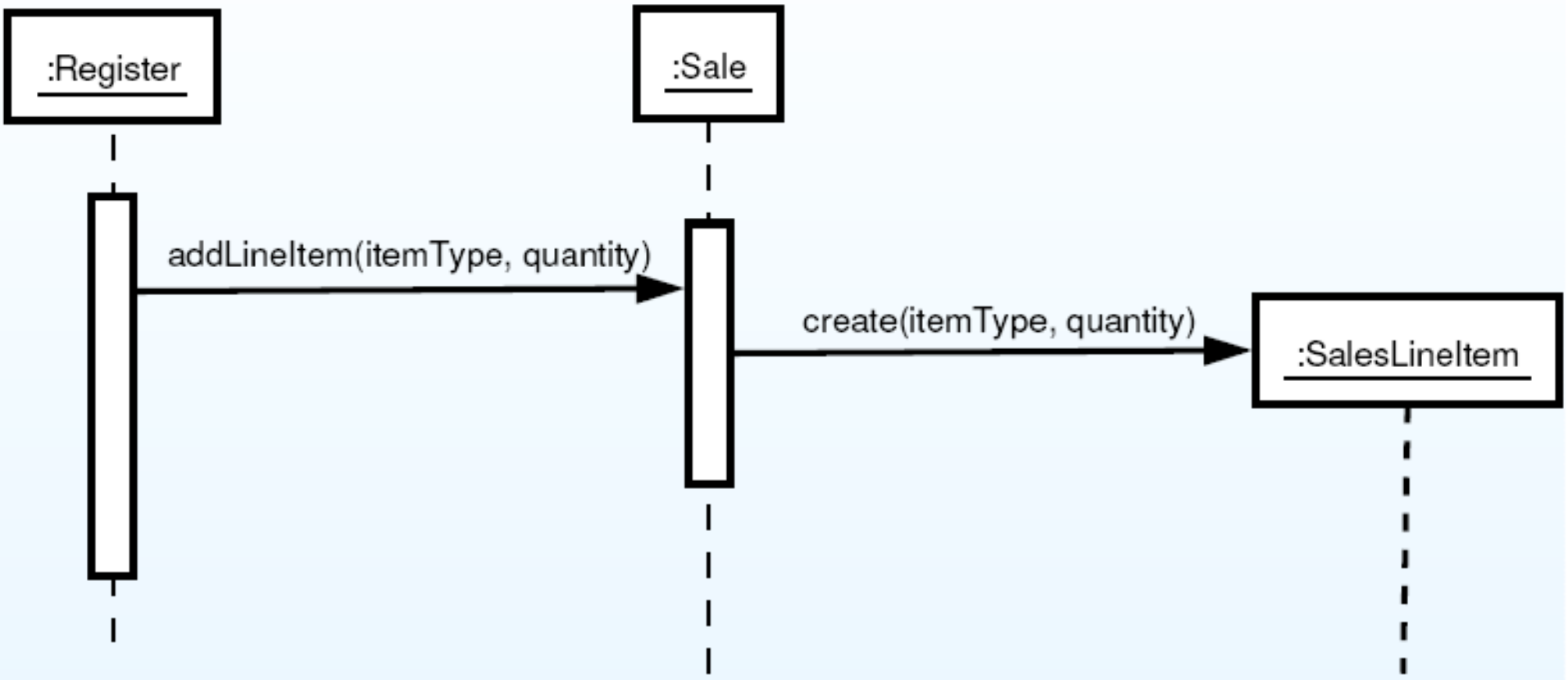
Creator 2

- Who is responsible for creating an instance of the class SalesLineItem?



Creator 3

- Because Sale contains (aggregates) instance type SalesLineItem, it is a good candidate to be assigned the responsibility for creating these instances



Low coupling

- The coupling is a measure of the degree of dependence of a class on other classes
- Types of Dependence:
  - It is connected to
  - It has knowledge about
  - It is based on
- A class that has low coupling (reduced) does not depend on "many" other classes; where "many" depends on context
- A class that has high coupling depends on many other classes

Coupling 2

- Problems caused by coupling:
  - changes in the related classes force local changes
  - difficult to understand classes in isolation (out of context)
  - hard to reuse classes because their use requires the presence of dependent classes

Coupling 3

- Common forms of coupling from class A to class B are:
  - A has an attribute of type B
  - An instance of the class A calls a service offered by an object of type B
  - A has a method that references B (parameter, local object, the object returned)
  - A is a subclass (direct or indirect) of B
  - B is an interface, and A implements this interface

Law of Demeter

- *Don't talk to strangers*

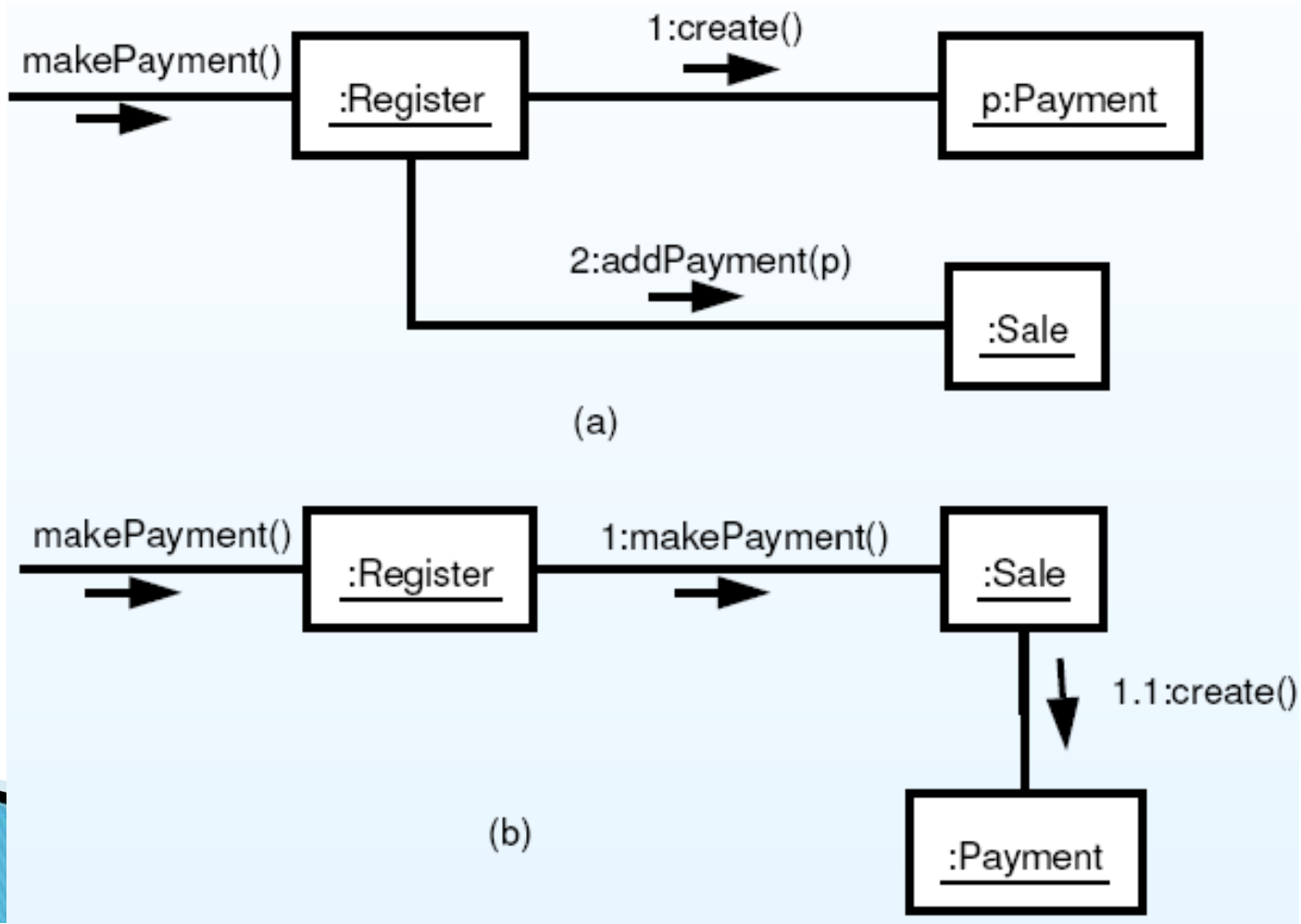
- Any method of an object should call only methods belonging to:
  - himself
  - any parameter of the method
  - any object that it created
  - any objects that it contains

The visualization of couplings

- Class Diagram
- Collaboration Diagram

Example 1

- There are links between all classes
- It eliminates coupling between Register and Payment



High Cohesion

- **Cohesion** is a measure of how strong the responsibilities of a class are focused
- A class whose responsibilities are very closely linked and which is not very much has a **great cohesion**
- A class that does many things that are not related to each other or that does too many things has a **low (weak) cohesion**

Cohesion

- Problems caused by a weak cohesion:
  - hard to understand
  - hard to reused
  - hard to maintain
  - delicate; such classes are always subject to change

Cohesion and coupling

- Are old principles in software design
- Promote a modular design
- Modularity is the property of a system that has been decomposed into a set of cohesive and loosely coupled modules

Controller 1

- **Problem:** Who is responsible for dealing with an event generated by an actor?
- These events are associated with operations of the system
- A Controller is an object that does not belong to the graphical interface, which is responsible for receiving or managing an event
- A controller defines a method for a corresponding operation of the system

Controller 2

- **Solution:** assign the responsibility for receiving or managing an event to a class representing one of the following choices:
  - Represents the entire system or subsystem (facade controller)
  - It is a usage scenario in which the event occurs

Controller 3

- Normally a controller should delegate to other objects the work that is to be done
- The controller coordinates and controls the activity, but it does not do too many things himself
- A common mistake in the design of a controller is to assign it too many responsibilities (facade controller)

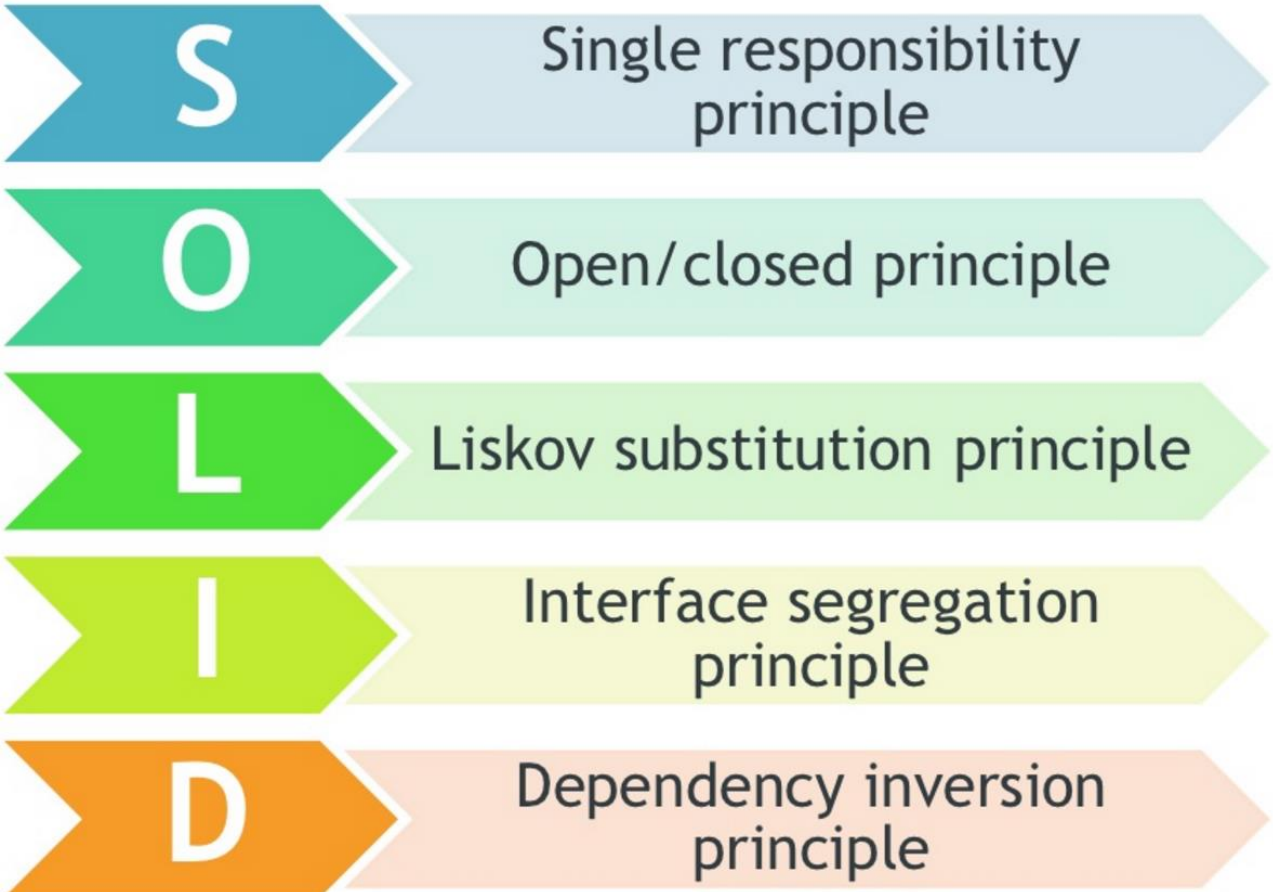
SOLID and Other Principles

- SOLID Principles
  - SRP – Single Responsibility Principle
  - OCP – Open/Closed Principle
  - LSP – Liskov Substitution Principle
  - ISP – Interface Segregation Principle
  - DIP – Dependency Inversion Principle
- DRY – Don't Repeat Yourself
- YAGNI – You Aren't Gonna Need It
- KISS – Keep It Simple, Stupid

S.O.L.I.D.
Principles

SOLID

- SOLID was introduced by Robert C. Martin in the an article called the “Principles of Object Oriented Design” in the early 2000s



SOLID – Single Responsibility Principle

- Every object should have a single responsibility, and all its services should be narrowly aligned with that responsibility

SOLID – SRP – Definitions

- "The Single Responsibility Principle states that every object should have a single responsibility, and that responsibility should be entirely encapsulated by the class." – Wikipedia
- "There should never be more than one reason for a class to change." – Robert Martin
- Low coupling & strong cohesion

SOLID – SRP – Problems & Solutions

- Classic violations
  - Objects that can print/draw themselves
  - Objects that can save/restore themselves
- Classic solution
  - Separate printer & Separate saver
- Solution
  - Multiple small interfaces (ISP)
  - Many small classes
  - Distinct responsibilities
- Result
  - Flexible design
  - Lower coupling & Higher cohesion

SOLID – SRP – Example

- Two responsabilities

```
interface Modem {
 public void dial(String pno);
 public void hangup();

 public void send(char c);
 public char recv();
}
```

- Separated interfaces

```
interface DataChannel {
 public void send(char c);
 public char recv();
}
```

```
interface Connection {
 public void dial(String phn);
 public char hangup();
}
```

SOLID – Open/Closed Principle

- *Open chest surgery is not needed when putting on a coat*
- Bertrand Meyer originated the OCP term in his 1988 book, *Object Oriented Software Construction*

SOLID – OCP – Definitions

- "The Open / Closed Principle states that software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification." – Wikipedia
- "All systems change during their life cycles. This must be borne in mind when developing systems expected to last longer than the first version." – Ivar Jacobson
- **Open** to Extension – New behavior can be added in the future
- **Closed** to Modification – Changes to source or binary code are not required

SOLID – OCP – How?

- Change behavior without changing code?!
  - Rely on abstractions, not implementations
  - Do not limit the variety of implementations
- In .NET – Interfaces, Abstract Classes
- In procedural code – Use parameters
- Approaches to achieve OCP
  - Parameters – Pass delegates / callbacks
  - Inheritance / Template Method pattern – Child types override behavior of a base class
  - Composition / Strategy pattern – Client code depends on abstraction, "Plug in" model

SOLID – OCP – Problems & Solutions

- Classic violations
  - Each change requires re-testing (possible bugs)
  - Cascading changes through modules
  - Logic depends on conditional statements
- Classic solution
  - New classes (nothing depends on them yet)
  - New classes (no legacy coupling)
- When to apply OCP?
  - Experience tell you
- OCP add complexity to design (TANSTAAFL)
- No design can be closed against all changes

SOLID – OCP – Example

```
// Open-Close Principle - Bad example
class GraphicEditor {
 public void drawShape(Shape s) {
  if (s.m_type==1)
        drawRectangle(s);
  else if (s.m_type==2)
        drawCircle(s);
 }
 public void drawCircle(Circle r)
{....}
 public void drawRectangle(Rectangle r)
{....}
}

class Shape {
 int m_type;
}

class Rectangle extends Shape {
 Rectangle() {super.m_type=1;}
}

class Circle extends Shape {
 Circle() {super.m_type=2;}
}
```

```
// Open-Close Principle - Good
example
class GraphicEditor {
 public void drawShape(Shape s) {
        s.draw();
 }
}

class Shape {
        abstract void draw();
}

class Rectangle extends Shape {
 public void draw() {
        // draw the rectangle
 }
}
```

SOLID – Liskov Substitution

- If it looks like a duck, quacks like a duck, but needs batteries – you probably have the wrong abstraction
- Barbara Liskov described the principle in 1988

SOLID – LSP – Definitions

- "The Liskov Substitution Principle states that Subtypes must be substitutable for their base types." – Agile Principles, Patterns, and Practices in C#
- Substitutability – child classes must not
  - Remove base class behavior
  - Violate base class invariants
- Normal OOP inheritance
  - IS-A relationship
- Liskov Substitution inheritance
  - IS-SUBSTITUTABLE-FOR

SOLID – LSP – Problems & Solutions

- The problem
  - Polymorphism break Client code expectations
  - "Fixing" by adding if-then – nightmare (OCP)
- Classic violations
  - Type checking for different methods
  - Not implemented overridden methods
  - Virtual methods in constructor
- Solutions
  - “Tell, Don’t Ask” – Don’t ask for types and Tell the object what to do
  - Refactoring to base class- Common functionality and Introduce third class

SOLID – LSP – Example (1)

```
// Violation of Liskov's Substitution Principle
 class Rectangle{
   int m_width;
   int m_height;

   public void setWidth(int width){
         m_width = width;
   }

   public void setHeight(int h){
         m_height = ht;
   }

   public int getWidth(){
         return m_width;
   }

   public int getHeight(){
         return m_height;
   }

   public int getArea(){
         return m_width * m_height;
   }
 }
```

```
class Square extends Rectangle {
  public void setWidth(int width){
        m_width = width;
        m_height = width;
  }

  public void setHeight(int height){
        m_width = height;
        m_height = height;
  }
}
```

SOLID – LSP – Example (2)

```
class LspTest
{
private static Rectangle getNewRectangle()
{
        // it can be an object returned by some factory ...
        return new Square();
}

public static void main (String args[])
{
        Rectangle r = LspTest.getNewRectangle();
        r.setWidth(5);
        r.setHeight(10);
// user knows that r it's a rectangle. It assumes that he's able to set the width
 and height as for the base class

        System.out.println(r.getArea());
        // now he's surprised to see that the area is 100 instead of 50.
}
}
```

SOLID – Interface Segregation

- You want me to plug this in. Where?

SOLID – ISP – Definitions

- “The Interface Segregation Principle states that Clients should not be forced to depend on methods they do not use.” – Agile Principles, Patterns, and Practices in C#
- Prefer small, cohesive interfaces – Interface is the interface type + All public members of a class
- Divide "fat" interfaces into smaller ones
  - “fat” interfaces means classes with useless methods, increased coupling, reduced flexibility and maintainability

SOLID – ISP – Problems & Solutions

- Classic violations
  - Unimplemented methods (also in LSP)
  - Use of only small portion of a class
- When to fix?
  - Once there is pain! Do not fix, if is not broken!
  - If the "fat" interface is yours, separate it to smaller ones
  - If the "fat" interface is not yours, use "Adapter" pattern
- Solutions
  - Small interfaces
  - Cohesive interfaces
  - Focused interfaces
  - Let the client define interfaces
  - Package interfaces with their implementation

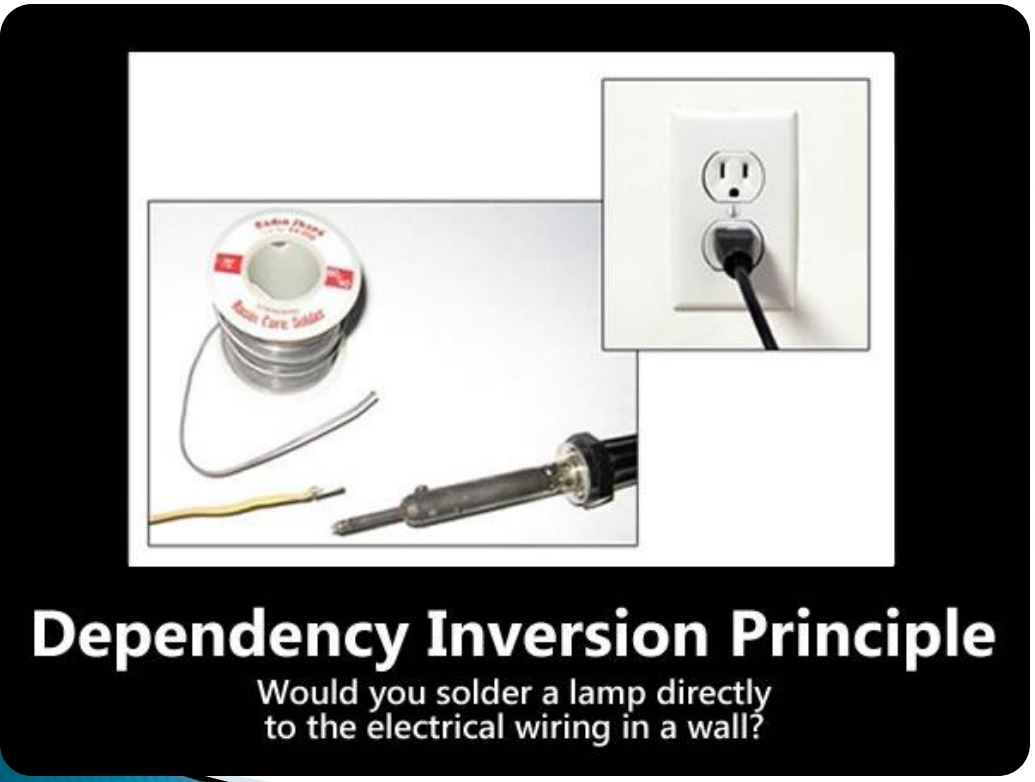
SOLID – ISP – Example

```
//Bad example (polluted interface)


interface Worker {
 void work();
 void eat();
}

ManWorker implements Worker {
        void work() {…};
        void eat() {30 min break;};
}


RobotWorker implements Worker {
        void work() {…};
        void eat() {//Not Appliciable
                for a RobotWorker};
}
```

```
//Solution: split into two interfaces


interface Workable {
        public void work();
 }

 interface Feedable{
        public void eat();
 }
```

SOLID – Dependency Inversion

- Would you solder a lamp directly to the electrical wiring in a wall?



SOLID – DIP – Definitions

- "High-level modules should not depend on low-level modules. Both should depend on abstractions."

- "Abstractions should not depend on details. Details should depend on abstractions." – Agile Principles, Patterns, and Practices in C#

SOLID – DIP – Dependency

- Framework
- Third Party Libraries
- Database
- File System
- Email
- Web Services
- System Resources (Clock)
- Configuration
- The new Keyword
- Static methods
- Thread.Sleep
- Random

SOLID – DIP – Problems & Solutions

- How it should be
  - Classes should declare what they need
  - Constructors should require dependencies
  - Dependencies should be abstractions and be shown
- How to do it
  - Dependency Injection
  - The Hollywood principle "Don't call us, we'll call you!"
- Classic violations
  - Using of the new keyword, static methods/properties
- How to fix?
  - Default constructor, main method/starting point
  - Inversion of Control container

SOLID – DIP – Example

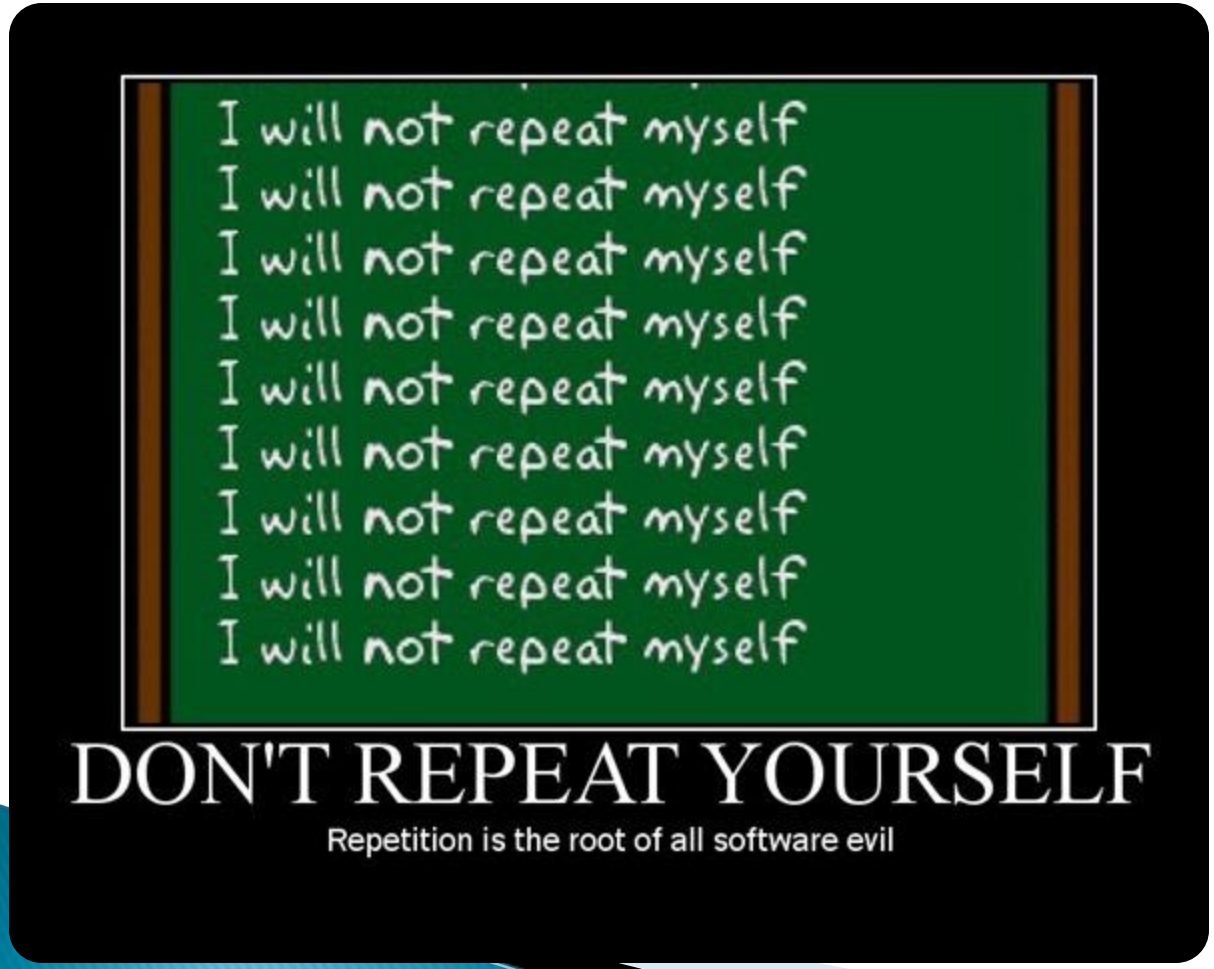
```
//DIP - bad example
public class EmployeeService {
        private EmployeeFinder emFinder //concrete class, not abstract. Can access a SQL DB for instance
        public Employee findEmployee(…) {
                emFinder.findEmployee(…)
        }
}

//DIP - fixed
public class EmployeeService {
        private IEmployeeFinder emFinder //depends on an abstraction, no an implementation
        public Employee findEmployee(…) {
                emFinder.findEmployee(…)
        }
}
//Now its possible to change the finder to be a XmlEmployeeFinder, DBEmployeeFinder, FlatFileEmployeeFinder,
MockEmployeeFinder….
```

Other Principles

- Don't Repeat Yourself (DRY)
- You Ain't Gonna Need It (YAGNI)
- Keep It Simple, Stupid (KISS)

OP – Don't Repeat Yourself

- Repetition is the root of all software evil



OP – DRY – Definitions

- "Every piece of knowledge must have a single, unambiguous representation in the system.“ – The Pragmatic Programmer
- "Repetition in logic calls for abstraction. Repetition in process calls for automation.“ – 97 Things Every Programmer Should Know
- Variations include:
  - Once and Only Once
  - Duplication Is Evil (DIE)

OP – DRY – Problems

- Magic Strings/Values
- Duplicate logic in multiple locations
- Repeated if-then logic
- Conditionals instead of polymorphism
- Repeated Execution Patterns
- Lots of duplicate, probably copy-pasted, code
- Only manual tests
- Static methods everywhere

OP – You Ain't Gonna Need It

- Don’t waste resources on what you might need

OP – YAGNI – Definitions

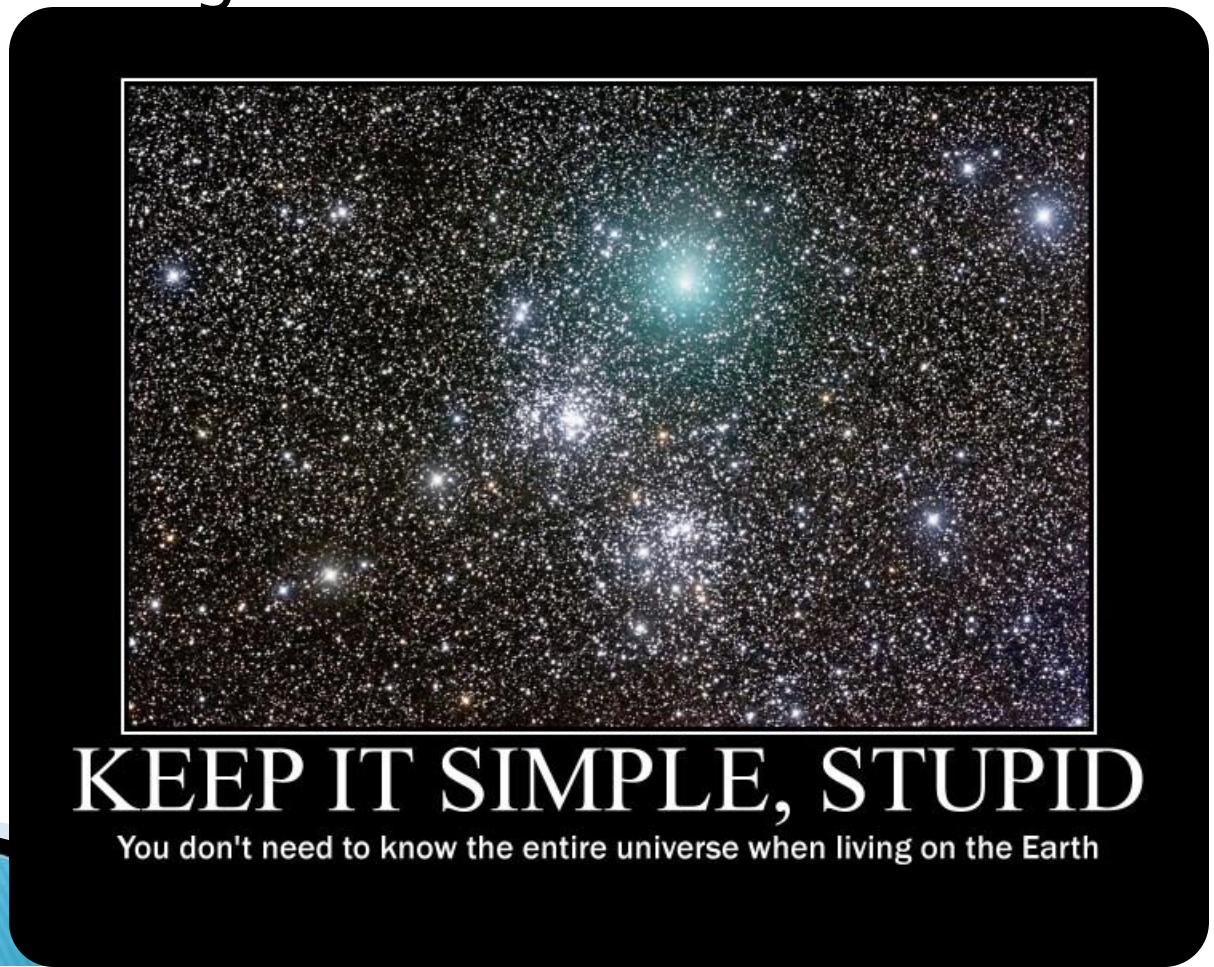
- "A programmer should not add functionality until deemed necessary.“ – Wikipedia
- "Always implement things when you actually need them, never when you just foresee that you need them.“ – Ron Jeffries, XP co-founder

OP – YAGNI – Problems

- Time for adding, testing, improving
- Debugging, documented, supported
- Difficult for requirements
- Larger and complicate software
- May lead to adding even more features
- May be not know to clients

OP – Keep It Simple, Stupid

- You don’t need to know the entire universe when living on the Earth



OP – KISS – Definitions

- "Most systems work best if they are kept simple.“ – U.S. Navy
- "Simplicity should be a key goal in design and unnecessary complexity should be avoided.“ – Wikipedia

Conclusions

- SOLID
- GRASP

Questions

- 1) Argue for the using of SOLID.
- 2) Argue for the using of diagrams.
- 3) Come with motivations for not using diagrams or SOLID.
- 4) How can we use information relating to cohesion and coupling? When evaluating a project. When evaluating a team member.

- Criticism: http://sourcemaking.com/design_patterns

Bibliography

- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software* (GangOfFour)
- Ovidiu Gheorghieş, Curs 7 IP
- Adrian Iftene, Curs 9 TAIP: http://thor.info.uaic.ro/~adiftene/Scoala/2011/TAIP/Courses/TAIP09.pdf

Bibliography

- Craig Larman. *Applying UML and Patterns. An Introduction to Object Oriented Analysis and Design*

- Ovidiu Gheorghieș, Curs 6 IP

Links (1)

- WebProjectManager: http://profs.info.uaic.ro/~adrianaa/uml/
- Diagrame de Stare și de Activitate: http://software.ucv.ro/~soimu_anca/itpm/Diagrame%20de%20Stare%20si%20Activitate.doc
- Deployment Diagram: http://en.wikipedia.org/wiki/Deployment_diagram http://www.agilemodeling.com/artifacts/deploymentDiagram.htm
- GRASP: http://en.wikipedia.org/wiki/GRASP_(Object_Oriented_Design)
- http://web.cs.wpi.edu/~gpollice/cs4233-a05/CourseNotes/maps/class4/GRASPpatterns.html
- Introduction to GRASP Patterns: http://faculty.inverhills.edu/dlevitt/CS%202000%20(FP)/GRASP%20Patterns.pdf

Links (2)

- Gang-Of-Four: http://c2.com/cgi/wiki?GangOfFour, http://www.uml.org.cn/c%2B%2B/pdf/DesignPatterns.pdf
- Design Patterns Book: http://c2.com/cgi/wiki?DesignPatternsBook
- About Design Patterns: http://www.javacamp.org/designPattern/
- Design Patterns – Java companion: http://www.patterndepot.com/put/8/JavaPatterns.htm
- Java Design patterns: http://www.allapplabs.com/java_design_patterns/java_design_patterns.htm
- Overview of Design Patterns: http://www.mindspring.com/~mgrand/pattern_synopses.htm
- Gang of Four: http://en.wikipedia.org/wiki/Gang_of_four
- JUnit in Eclipse: http://www.vogella.de/articles/JUnit/article.html
- JUnit in NetBeans: http://netbeans.org/kb/docs/java/junit-intro.html

Links (3)

- https://scotch.io/bar-talk/s-o-l-i-d-the-first-five-principles-of-object-oriented-design
- https://www.slideshare.net/enbohm/solid-design-principles-9016117
- https://siderite.blogspot.com/2017/02/solid-principles-plus-dry-yagni-kiss-final.html
- https://thefullstack.xyz/dry-yagni-kiss-tdd-soc-bdfu